TIC-TAC-TOE GAME USING PYTHON

AIM

To design and develop an interactive Tic-Tac-Toe game using Python Tkinter. Two players take turns marking X and O on a 3 × 3 board. The program checks for a winning combination or a draw and provides restart and exit options.

PROCEDURE

1. Import the tkinter library and the message box module.
2.
2. Create the main application window and set its title, size behaviour, and background colour.

3. Create a 3 × 3 board using nine Tkinter buttons.

4. Store the state of the nine cells in a list and set Player X to start.

5. Use a button command to place the current player's mark in the selected empty cell.

6. After each move, check all rows, columns, and diagonals for a winning combination.

7. If no player has won and all cells are occupied, declare a draw.

8. Switch between players X and O after every valid move.

9. Add a Restart Game button to clear the board and an Exit button to close the window.

10. Run the program and test normal moves, wins, draws, and restarting

PROGRAM CODE

```
import tkinter as tk
from tkinter import messagebox

# Create the main window
root = tk.Tk()
root.title("TIC-TAC-TOE")
root.resizable(False, False)
root.configure(bg="#172554")

# Game variables
current_player = "X"
game_over = False
board = [""] * 9
buttons = []

def check_winner():
    """Return X or O if a player has won; return None otherwise."""
    winning_lines = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8),  # Rows
        (0, 3, 6), (1, 4, 7), (2, 5, 8),  # Columns
        (0, 4, 8), (2, 4, 6)              # Diagonals
    ]

    for a, b, c in winning_lines:
        if board[a] == board[b] == board[c] and board[a] != "":
            return board[a]
    return None

def play_turn(index):
    global current_player, game_over

    # Ignore clicks if the game has ended or the cell is occupied
    if game_over or board[index] != "":
        return

    board[index] = current_player
    buttons[index].config(
        text=current_player,
        state="disabled",
        disabledforeground="#FACC15" if current_player == "X" else
"#86EFAC"
    )

    winner = check_winner()

    if winner:
        status_label.config(text=f"Player {winner} wins!")
        game_over = True
        messagebox.showinfo("Game Over", f"Player {winner} wins!")
        return
```

```
    if "" not in board:
        status_label.config(text="It's a draw!")
        game_over = True
        messagebox.showinfo("Game Over", "It's a draw!")
        return

    current_player = "O" if current_player == "X" else "X"
    status_label.config(text=f"Player {current_player}'s turn")

def restart_game():
    global current_player, game_over, board

    current_player = "X"
    game_over = False
    board = [""] * 9

    for button in buttons:
        button.config(text="", state="normal", fg="white")

    status_label.config(text="Player X's turn")

# Heading
title_label = tk.Label(
    root,
    text="TIC-TAC-TOE",
    font=("Arial Black", 22),
    bg="#172554",
    fg="white"
)
title_label.pack(pady=(14, 6))

status_label = tk.Label(
    root,
    text="Player X's turn",
    font=("Arial", 14, "bold"),
    bg="#172554",
    fg="#FACC15"
)
status_label.pack(pady=6)

# Game board
board_frame = tk.Frame(root, bg="#172554")
board_frame.pack(padx=18, pady=12)

for i in range(9):
    button = tk.Button(
        board_frame,
        text="",
        font=("Arial Black", 24),
        width=4,
        height=2,
        bg="#1E3A8A",
```

```
        fg="white",
        activebackground="#2563EB",
        relief="raised",
        command=lambda index=i: play_turn(index)
    )
    button.grid(row=i // 3, column=i % 3, padx=4, pady=4)
    buttons.append(button)

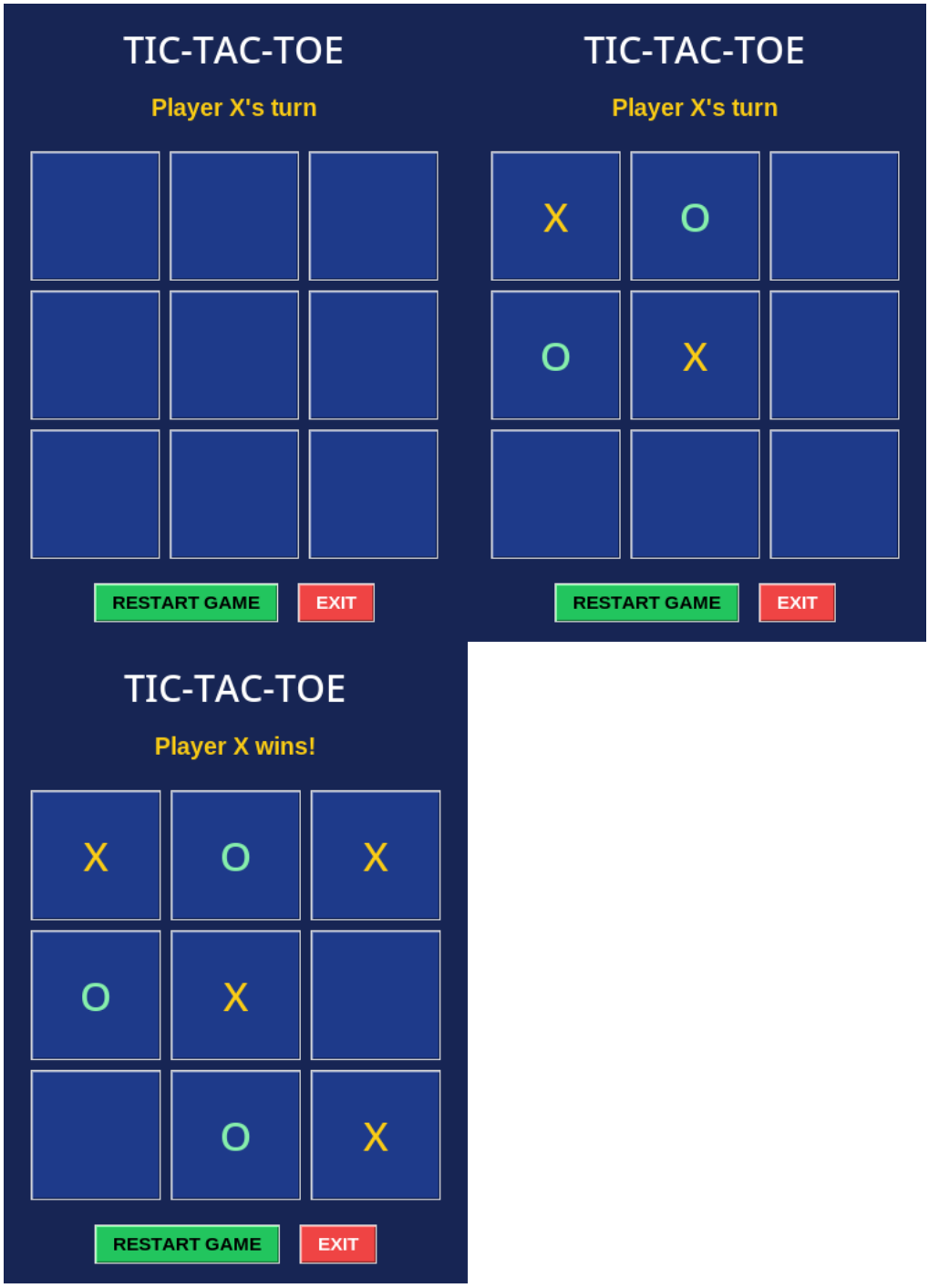
# Restart and exit controls
control_frame = tk.Frame(root, bg="#172554")
control_frame.pack(pady=(4, 16))

tk.Button(
    control_frame,
    text="RESTART GAME",
    font=("Arial", 11, "bold"),
    bg="#22C55E",
    fg="black",
    command=restart_game
).pack(side="left", padx=8)

tk.Button(
    control_frame,
    text="EXIT",
    font=("Arial", 11, "bold"),
    bg="#EF4444",
    fg="white",
    command=root.destroy
).pack(side="left", padx=8)

root.mainloop()
```

output:



RESULT:

Thus, an interactive Tic tac Toe was developed using Python Tkinter. The program demonstrates event-driven programming, GUI design, conditional statements, and basic game development concepts.